DESIGN PATTERNS

PROXY DESIGN PATTERN

Controlling Access. Adding Intelligence.

A structural design pattern that provides a **surrogate** or placeholder for another object to **control access** to it, improve **performance**, and add **security**.

USED EVERYWHERE

- **Security & Access Control**
- **Lazy Loading & Virtualization**
- **Caching & Performance**
- **Remote Communication**
- **Logging & Monitoring**



Same Interface. Controlled Access.

LECTURE 5 OF DESIGN PATTERNS SERIES

After: Singleton • Factory • Facade • Adapter

PROXY DESIGN PATTERN

*Same interface, but **controlled access.***



WHERE ARE WE?

- **Singleton** — one object
- **Factory** — object creation
- **Facade** — simplified interface
- **Adapter** — interface conversion

Now we add a new idea...

PROXY

Same interface, but **controlled access.**



Controls access to the real object

- Access Control
- Lazy Loading
- Logging
- Caching
- Security

The actual object (resource)

Key Idea

The client talks to the proxy using the **same interface.** The proxy decides whether to forward the request.

PROXY PATTERN | Provide a surrogate or placeholder to control access to another object.

PROXY DESIGN PATTERN – Introduction

*Same interface, but **controlled access.***

1. INTRODUCE THE PROBLEM

Let's start with a story...
Imagine your **university server**.

Students cannot directly access

- Student database
- Finance records
- Exam paper repository

Instead

Student → Authentication System → University Database

The authentication system decides

- Is the student logged in?
- Is the student allowed?
- Which records can be viewed?

This authentication system is acting as a **Proxy**.

University Server Access Scenario



Student

Makes a request (using the normal interface)

Authentication System (PROXY)

- Check login
- Check permissions
- Decide what to allow
- Forward the request

Acts as an intermediary. Controls access to the real resources.

University Database (Real Subject)

- Student Database
- Finance Records
- Exam Paper Repository

Actual data and resources are stored and managed here.

Students talk to the system using the **same interface**, but the proxy **decides what can be accessed** and **what cannot**.

PROXY DESIGN PATTERN – Everyday Examples

*Same interface, but **controlled access.***

EVERYDAY PROXY EXAMPLES

Real Life	Proxy
 	**Security guard** Controls access to building
 	**Receptionist** Controls visitors
 	**Credit card** Represents your bank account
 	**Hotel receptionist** Gives room only after verification
 	**Ticket checker** Allows entry only with valid ticket
 	**Parent signing school form** Acts on behalf of child

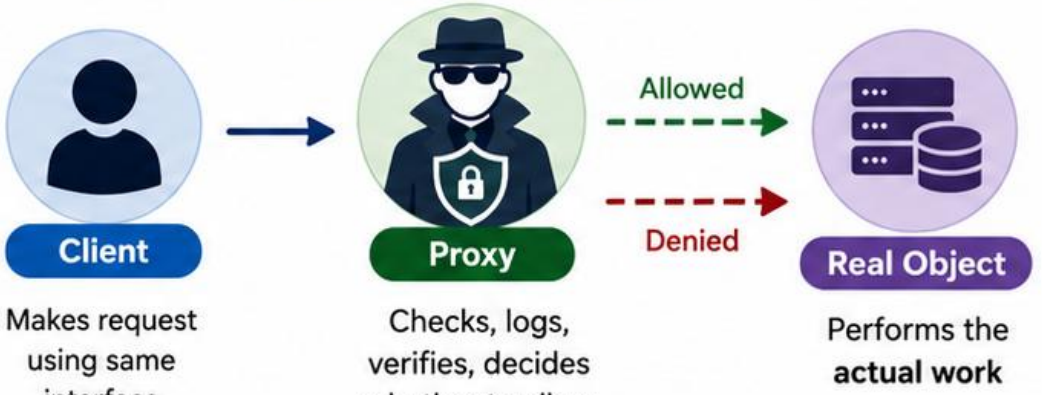
ASK

Which one actually **performs the work**?

ANSWER

The **real object**.
The proxy only **decides** whether
the request should continue.

HOW IT WORKS



The proxy looks exactly like the real object to the client (same interface),
but it **controls, protects, and manages access** before the real object does the work.

PROXY DESIGN PATTERN

DEFINITION

Provides a **placeholder** or **representative** object that **controls access** to another object.

THE PROXY

- **Validates** requests
- **Delays** object creation
- **Caches** results
- **Records** activity
- **Forwards** requests

HOW IT WORKS



The client **cannot distinguish** whether it communicates with the **proxy** or the **real object**.

KEY IDEA

The proxy has the **same interface** as the real object. Clients use it in the same way, but the proxy **decides how and when** the real object is accessed.

- Security
- Lazy Loading
- Caching
- Logging
- Remote Access

Proxy acts as an intermediary that **controls access** while keeping the same interface for the client.

PROXY DESIGN PATTERN

*Why not call the object **directly**?*

WITHOUT PROXY



PROBLEMS

- No **authentication**
- No **logging**
- No **caching**
- Expensive object **always created**

VS

WITH PROXY



BENEFITS

- **Authentication**
- **Logging**
- **Caching**
- **Lazy loading** (create object only when needed)

Using a **Proxy** gives you **control**, **protection**, and **performance**.

It acts as an **intermediary** between the client and the real object.

PROXY DESIGN PATTERN – Modern Software Uses

*Proxies are used in many different forms to **control access**, **improve performance**, and **protect resources** in modern applications.*

Proxy		Purpose	Example
Virtual Proxy (Lazy Loading)		Delays creation of the real object until it is actually needed. **Lazy loading**	Load image only when it becomes visible on the screen.
Protection Proxy (Authorization)		Controls access to the real object by checking permissions and **authorization**.	Bank account – allow access only to the authorized account holder.
Remote Proxy (Remote Server)		Represents an object that is located in a **different address space** (e.g., another server).	Access REST API on a remote server as if it were local.
Cache Proxy (Caching)		Stores results of expensive operations and returns **cached** results for future requests.	Browser cache – store web responses and serve quickly next time.
Logging Proxy (Logging)		Logs requests and responses for **monitoring**, **auditing**, or **debugging**.	Record all payment requests and responses for audit (payment logs).
Smart Proxy (Reference Counting)		Manages **reference counting** and performs additional management tasks.	Memory management – release object when no longer referenced.

Proxies add a layer of control between the client and the real object without changing the interface.
They improve **security**, **performance**, **reliability**, and **maintainability**.

REMOTE PROXY EXAMPLE – REST API & MICROSERVICES

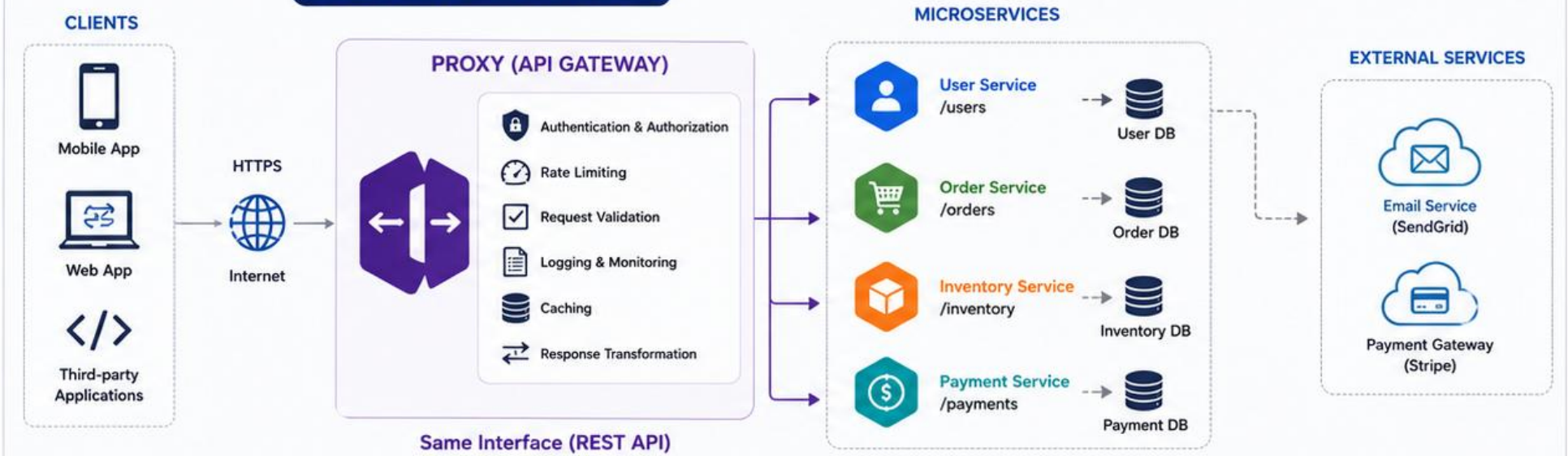
The **Proxy** represents a remote object and **controls access** to services over a network.

SCENARIO

A mobile app uses a backend microservices system through a **REST API**.

- The client **does not** call services directly.
- A **Proxy** (API Gateway) sits in front of the services.
- The proxy handles cross-cutting **concerns** and forwards the request to the right service.
- The service processes the request and the response returns back through the **Proxy** to the client.

ARCHITECTURE OVERVIEW



KEY IDEA

The client talks to the **Proxy** using the **same interface** (REST API). The proxy hides the complexity of remote services and **network**.

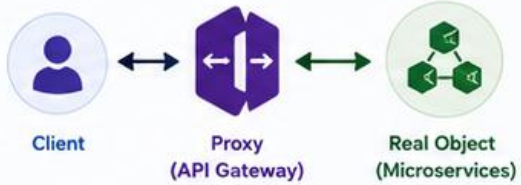
REQUEST FLOW (Example: Get /orders)



BENEFITS OF USING PROXY

- **Security** — Centralized authentication and authorization
- **Performance** — Caching, compression and rate limiting
- **Resilience** — Circuit breaker, retries and failover
- **Maintainability** — Clients are decoupled from backend services
- **Observability** — Centralized logging, monitoring and metrics

HOW THIS MAPS TO PROXY PATTERN



The **Proxy** implements the **same interface** (REST API) as the real services and **controls access** to them.

PATTERN TYPE: Remote Proxy

In modern systems, the **API Gateway** acts as a **Proxy**.

- Single Entry Point for all clients
- Hides Complexity of Microservices
- Enforces Policies and Security
- Improves Performance and Reliability
- Enables Scalable Cloud-Native Apps

PROTECTION PROXY EXAMPLE – Hospital System

The Proxy controls access to the real object based on user roles and permissions.

USERS (CLIENTS)



1. Receptionist
Logs in to the Hospital System

Role: RECEPTIONIST



2. Doctor
Logs in to the Hospital System

Role: DOCTOR



3. Administrator
Logs in to the Hospital System

Role: ADMINISTRATOR

ALL USERS CALL THE SAME METHOD

```
PatientService.viewPatient(patientId)
// Client does not know whether it is
calling the proxy or the real object.
```

HOW IT WORKS

CLIENT

Calls `viewPatient()`

PROXY (Access Control Proxy)

Proxy Checks:

- Is user authenticated?
- Does user have permission?
- Is role allowed to access patient data?

Allowed → Forward Request

Denied → Return Access Denied

REAL OBJECT (PatientService)

Performs the real work and returns patient data.

RESULTS

Doctor (DOCTOR)
✓ Allowed
Can view patient details.

Receptionist (RECEPTIONIST)
✗ Denied
Access denied.

Administrator (ADMINISTRATOR)
✓ Allowed
Can view patient details.

ROLE ACCESS MATRIX

Role	PatientService.viewPatient()
DOCTOR	✓ Allowed
RECEPTIONIST	✗ Denied
ADMINISTRATOR	✓ Allowed

INTERFACE (Same for Proxy and Real Object)

```
public interface PatientService {
  Patient viewPatient(String patientId);
}
```

KEY TAKEAWAY

The **Protection Proxy** controls access to the real object. Clients use the **SAME interface**, but the proxy decides whether the request should be allowed or denied based on **roles** and **permissions**.

Protection Proxy adds a layer of **security** between the client and the sensitive data/resources.

The real object (PatientService) only handles requests that are **allowed**.

REMOTE PROXY EXAMPLE – Mobile Banking App

Provides a local representative for an object that exists in a different address space (across the network).

SCENARIO

Mobile Banking App

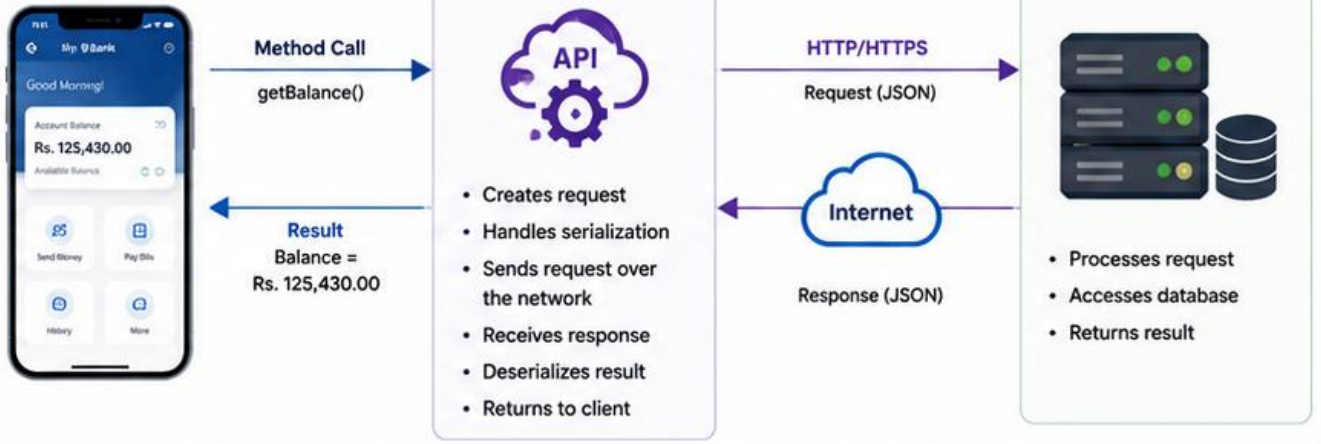
The banking app on your phone calls methods like **getBalance(), transfer(), payBill()**.

The app behaves as though the **Bank** object exists **locally**.

ACTUALITY

Every request goes through **network communication** to the remote bank server.

ARCHITECTURE OVERVIEW



THE ILLUSION FOR THE CLIENT



REQUEST-RESPONSE FLOW (Example: getBalance())



SAME INTERFACE FOR CLIENT

```
public interface BankService {
  double getBalance();
  void transfer(String to, double amount);
  List<Transsaction> getTransactions();
}
```

Client depends on the interface, not on local or remote implementation.

Proxy and Real Object implement the same interface.

KEY TAKEAWAY

- Remote Proxy acts as a local representative for an object in a remote server.
- Handles communication details (networking, serialization, deserialization).
- Hides the complexity of remote access from the client.
- Used heavily in REST APIs, gRPC, Web services, and microservices.

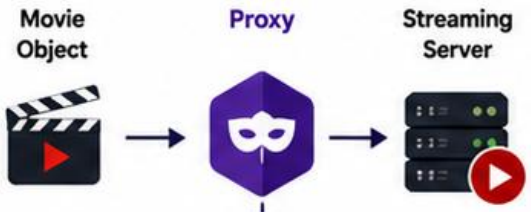


Remote Proxy = Communication Proxy | It forwards client requests to a remote object and returns the results, making remote objects feel local.

MODERN EXAMPLES OF PROXY DESIGN PATTERN

The Proxy acts as an intermediary that controls access to the real object.

Netflix



Proxy checks

- Checks subscription (Is user subscribed?)
- Checks country (Is content available?)
- Checks parental control (Allowed for profile?)

→ **Only then → Play movie**

What user sees

- Clicks Play
- Movie plays
- User doesn't know proxy is working in the background

Spotify



Proxy checks / handles

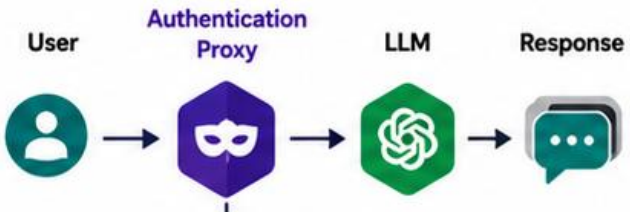
- Premium check (Free or Premium user?)
- Advertisement insertion (For free users)
- Cache songs (Store and serve faster)

→ **Only then → Stream song**

What user experiences

- Press Play
- Song streams
- Ads may play (if free user)
- Smooth experience (cached songs)

ChatGPT



Proxy handles

- Rate limiting (Too many requests?)
- Authentication (Is user valid?)
- Billing (Is plan active? / Usage tracking)

→ **Only then → Get response from LLM**

What user sees

- Types question
- Gets response
- Doesn't see the proxy checks happening behind the scenes

Google Drive



Virtual Proxy behavior

- Shows file thumbnail / metadata first
- Large file is NOT downloaded immediately
- Real file is downloaded only when user opens it

→ **Then → Open / Download file**

What user experiences

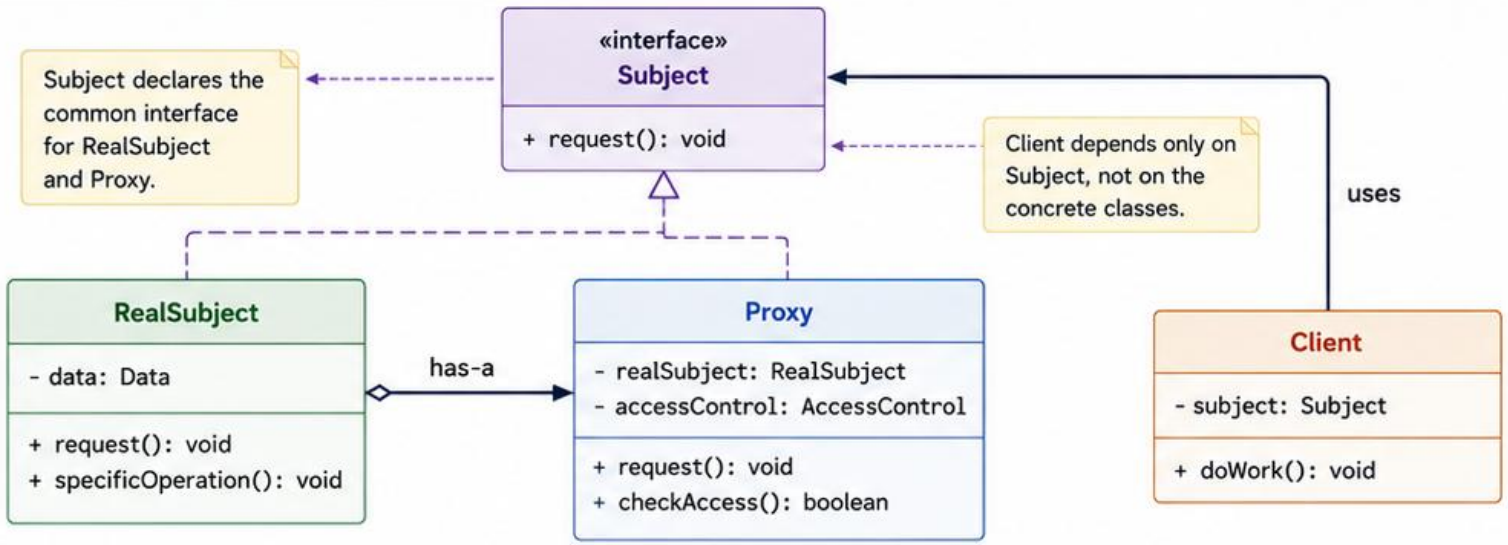
- Sees file list quickly (thumbnails)
- Clicks file
- File downloads and opens
- Faster and efficient (lazy loading)

Key Takeaway: The Proxy **controls access**, performs checks or optimizations, and then **forwards the request** to the real object.
The client communicates with the **Proxy**, but the **Proxy** communicates with the **Real Object**.

PROXY DESIGN PATTERN – UML EXPLANATION

The Proxy implements the same interface (Subject) as the RealSubject, so the client treats both objects in the same way.

PROXY DESIGN PATTERN – UML STRUCTURE



WHAT THIS MEANS

- Both Proxy and RealSubject implement the same Subject interface.
- The Client holds a reference of type Subject.
- At runtime, the object could be a Proxy or a RealSubject.
- The Client treats both objects uniformly through the Subject interface.

TYPICAL USAGE (PSEUDO CODE)

```
// Client code
Subject subject;
subject = new Proxy();      // or new RealSubject()
subject.request();          // Client doesn't know which one

// Client works with Subject only
void doWork() {
   subject.request();
}
```

QUESTION 1

Which classes implement Subject?

- Proxy
- RealSubject

ANSWER

Proxy and RealSubject implement Subject.

QUESTION 2

Why?

ANSWER

Because client should never know which one it receives.

Key Takeaway:

The Proxy and the RealSubject both implement the same Subject interface.
This ensures transparency and allows the Proxy to stand in for the RealSubject without the client knowing.

Client depends on abstractions, not concrete classes.

ADAPTER vs FACADE vs PROXY

*Three patterns. **Different goals.** Easy to confuse!*

Which one should I use?

Pattern	ADAPTER	FACADE	PROXY
Goal	**Convert interface** Allows incompatible interfaces to work together.	**Simplify interface** Provides a single, simple interface to a complex subsystem.	**Control access** Controls access to the real object.
Structure	Client → Target (Interface) → Adapter → Adaptee (Existing Class) *Adapter converts the interface of Adaptee to Target that the Client expects.*	Client → Facade → Subsystem A / Subsystem B / Subsystem C *Facade provides one simple interface to a set of subsystems.*	Client → Proxy → RealSubject (Real Object) *Proxy sits in front of the RealSubject and controls access to it.*
When to use	✔ When you want to use an existing class, but its interface does not match. ✔ You cannot change the existing class. ✔ Example: Using old payment gateway with new system.	✔ When the subsystem is complex and has many classes. ✔ You want to hide complexity and provide an easy-to-use interface. ✔ Example: Home theater system (one button to control many devices).	✔ When you need to control access to an object. ✔ When you want lazy loading, caching, logging, authentication, etc. ✔ Example: Virtual proxy for images, protection proxy for bank account.
Key Difference	Changes the **interface** so things can work together.	Hides the **complexity** by giving a **simplified** interface.	Controls the **access** to the real object.

SIDE-BY-SIDE EXAMPLE (Online Shopping)

ADAPTER

OldPaymentGateway has a different method. Adapter converts it to the new Payment interface.

Checkout → Payment (Interface) → PaymentAdapter → OldPayment Gateway

FACADE

Order process involves many steps. OrderFacade provides one simple method.

Client → OrderFacade → InventoryService / PaymentService / ShippingService

PROXY

Before accessing the real product service, Proxy checks login, cache, and logs.

Client → ProductService Proxy → ProductService (Real)

Quick Memory Tip

- **Adapter** is like a **plug adapter**: Makes different shapes fit.
- **Facade** is like **sunglasses**: Hides many things, shows it simply.
- **Proxy** is like a **security guard**: Controls who can go inside.

Remember:
Convert → Adapter
Simplify → Facade
Control → Proxy

PROXY DESIGN PATTERN – JAVA EXAMPLE

Control access to an object by placing a proxy in front of the real object.

SCENARIO: BANK LOGIN & WITHDRAWAL

After login, user wants to withdraw money. The proxy validates the request before allowing the real account to perform it.

CLASS DIAGRAM



The User deals with the Account interface. The Proxy decides whether to allow the request.

WHAT DOES THE PROXY CHECK?

1. **Password Authentication**
   Verifies the user's password.
2. **Daily Withdrawal Limit**
   Ensures the request does not exceed the daily limit.
3. **Account Status**
   Checks whether the account is active.
4. **Forward to RealAccount**
   If all checks pass, the request is forwarded to the real object.

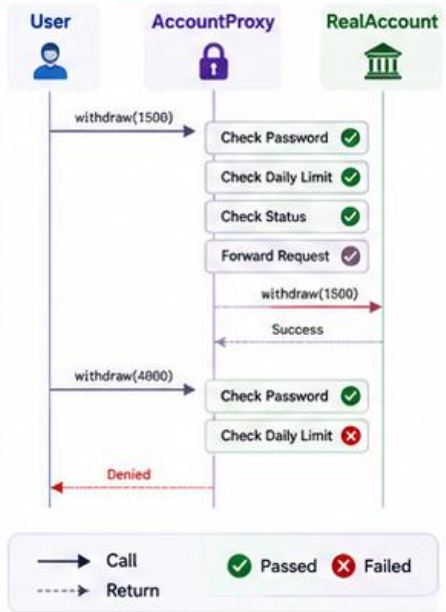
USAGE (MAIN OUTPUT)

```
[AccountProxy] Validating request...
[RealAccount] Processing withdrawal...
Withdrawal successful. New balance: 8500.0

[AccountProxy] Validating request...
Daily limit exceeded. Allowed: 3500.0

[AccountProxy] Validating request...
[RealAccount] Processing withdrawal...
Withdrawal successful. New balance: 7900.0
```

REQUEST FLOW



JAVA CODE

```java
// 1. Subject Interface
public interface Account {
    void withdraw(double amount);
}

// 2. Real Subject
class RealAccount implements Account {
    private String accountId;
    private double balance;

    public RealAccount(String accountId, double balance) {
        this.accountId = accountId;
        this.balance = balance;
    }

    @Override
    public void withdraw(double amount) {
        System.out.println("[RealAccount] Processing withdrawal...");
        if (balance >= amount) {
            balance -= amount;
            System.out.println("Withdrawal successful. New balance: " + balance);
        } else {
            System.out.println("Insufficient balance!");
        }
    }
}
```

```java
// 3. Proxy
class AccountProxy implements Account {
    private Account realAccount;
    private String password;
    private double dailyLimit;
    private double withdrawnToday = 0.0;
    private boolean active = true;

    public AccountProxy(Account realAccount, String password, double dailyLimit) {
        this.realAccount = realAccount;
        this.password = password;
        this.dailyLimit = dailyLimit;
    }

    // Business checks before forwarding
    @Override
    public void withdraw(double amount) {
        System.out.println("[AccountProxy] Validating request...");
        if (!authenticate(password)) return;
        if (!active) {
            System.out.println("Account is inactive. Contact bank.");
            return;
        }
        if (withdrawnToday + amount > dailyLimit) {
            System.out.println("Daily limit exceeded. Allowed: "
                + (dailyLimit - withdrawnToday));
            return;
        }
        withdrawnToday += amount;
        realAccount.withdraw(amount); // forward to RealAccount
    }

    private boolean authenticate(String pwd) {
        if ("bank123".equals(pwd)) {
            System.out.println("Invalid password!");
            return false;
        }
        return true;
    }
}
// 4. Client
class BankClient {
    public static void main(String[] args) {
        Account realAccount = new RealAccount("A1001", 10000.00);
        Account account = new AccountProxy(realAccount, "bank123", 5000.00);

        // User makes a withdrawal request
        account.withdraw(1500.00); // allowed
        account.withdraw(4000.00); // blocked (daily limit)
        account.withdraw(600.00);  // allowed
    }
```

KEY TAKEAWAY

The client talks to the Account (interface), not to RealAccount.
The AccountProxy controls access and ensures security and rules.
The client never knows whether it's talking to the proxy or the real object.

BENEFITS

- Controls access to sensitive operations
- Adds security, logging, caching, lazy loading, etc.
- Client code remains unchanged

PROXY DESIGN PATTERN – JAVA EXAMPLE 2

Virtual Proxy Example – Image Viewer (Lazy Loading)

SCENARIO

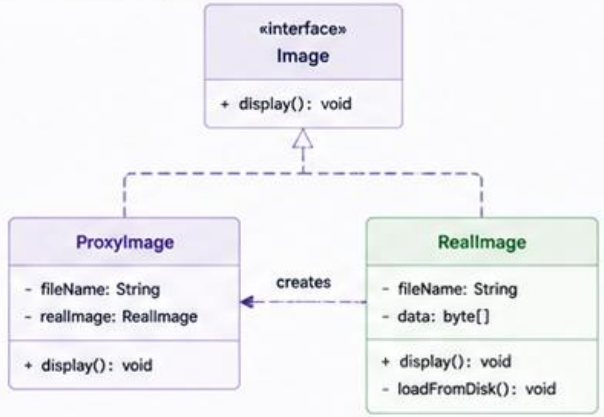
An image file is large.
We load it only when we really need to display it.
Until then, no memory or disk is used.

CONCEPT



Client talks to Image (interface). It may get ProxyImage or RealImage.
If the image is needed, ProxyImage creates RealImage and delegates.

CLASS DIAGRAM



CLIENT CODE

```java
// Client code
public class ImageViewerDemo {
    public static void main(String[] arps) {
        Image image = new ProxyImage("cat.jpg");
        System.out.println("ProxyImage created. Nothing loaded yet.");
        System.out.println("Calling display()...");
        image.display();       // Now the image will be loaded
        System.out.println("Calling display() again...");
        image.display();       // Already loaded, no reload
    }
}
```

```java
Image image = new ProxyImage("cat.jpg");
```

Nothing loads at this line.
Only when image.display() is called, RealImage is created and the file is loaded.

JAVA IMPLEMENTATION

1) Image (Subject Interface)

```java
public interface Image {
    void display();
}
```

2) RealImage (RealSubject)

```java
public class RealImage implements Image {
    private String fileName;
    private byte[] data;

    public RealImage(String fileName) {
        this.fileName = fileName;
        loadFromDisk();    // Expensive operation
    }

    private void loadFromDisk() {
        System.out.println("[RealImage] Loading " + fileName + " from disk...");
        try { Thread.sleep(1000); } catch (InterruptedException e) {}
        data = new byte[1024];  // Simulate large data
        System.out.println("[RealImage] Loaded " + fileName + " (" + data.length + " bytes)");
    }

    @Override
    public void display() {
        System.out.println("[RealImage] Displaying " + fileName);
    }
}
```

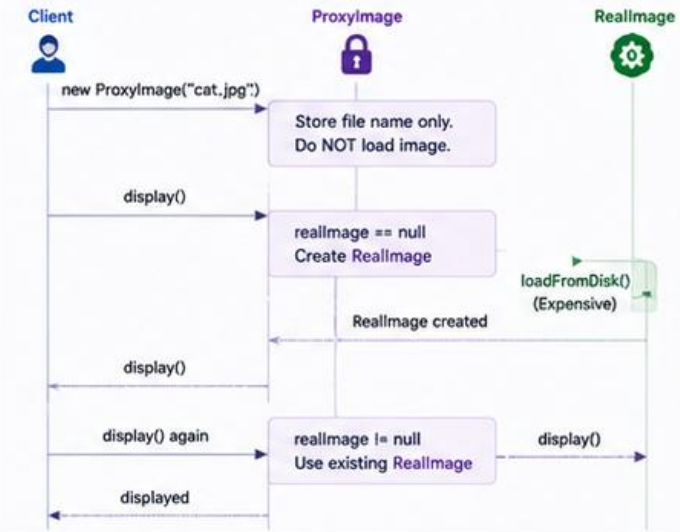
3) ProxyImage (Proxy)

```java
public class ProxyImage implements Image {
    private String fileName;
    private RealImage realImage;

    public ProxyImage(String fileName) {
        this.fileName = fileName;
        System.out.println("[ProxyImage] Created for " + fileName
                + " (nothing loaded yet)");
    }

    @Override
    public void display() {
        if (realImage == null) {
            System.out.println("[ProxyImage] Creating RealImage now...");
            realImage = new RealImage(fileName);
        } else {
            System.out.println("[ProxyImage] RealImage already created.");
        }
        realImage.display();
    }
}
```

WHAT HAPPENS (SEQUENCE)



SAMPLE OUTPUT (Console)

```
[ProxyImage] Created for cat.jpg (nothing loaded yet)
ProxyImage created. Nothing loaded yet.
Calling display()....
[ProxyImage] Creating RealImage now...
[RealImage] Loading cat.jpg from disk...
[RealImage] Loaded cat.jpg (1024 bytes)
[RealImage] Displaying cat.jpg
Calling display() again....
[ProxyImage] RealImage already created.
[RealImage] Displaying cat.jpg
```

KEY TAKEAWAY

- ProxyImage acts as a placeholder.
- RealImage is created only when the image is really needed.
- This saves memory, time, and resources.

Type: Virtual Proxy
Benefit: Lazy Loading of expensive objects

PROXY DESIGN PATTERN – SEQUENCE DIAGRAM

Flow of a request from Client → Proxy → Real Object → back to Client

PARTICIPANTS

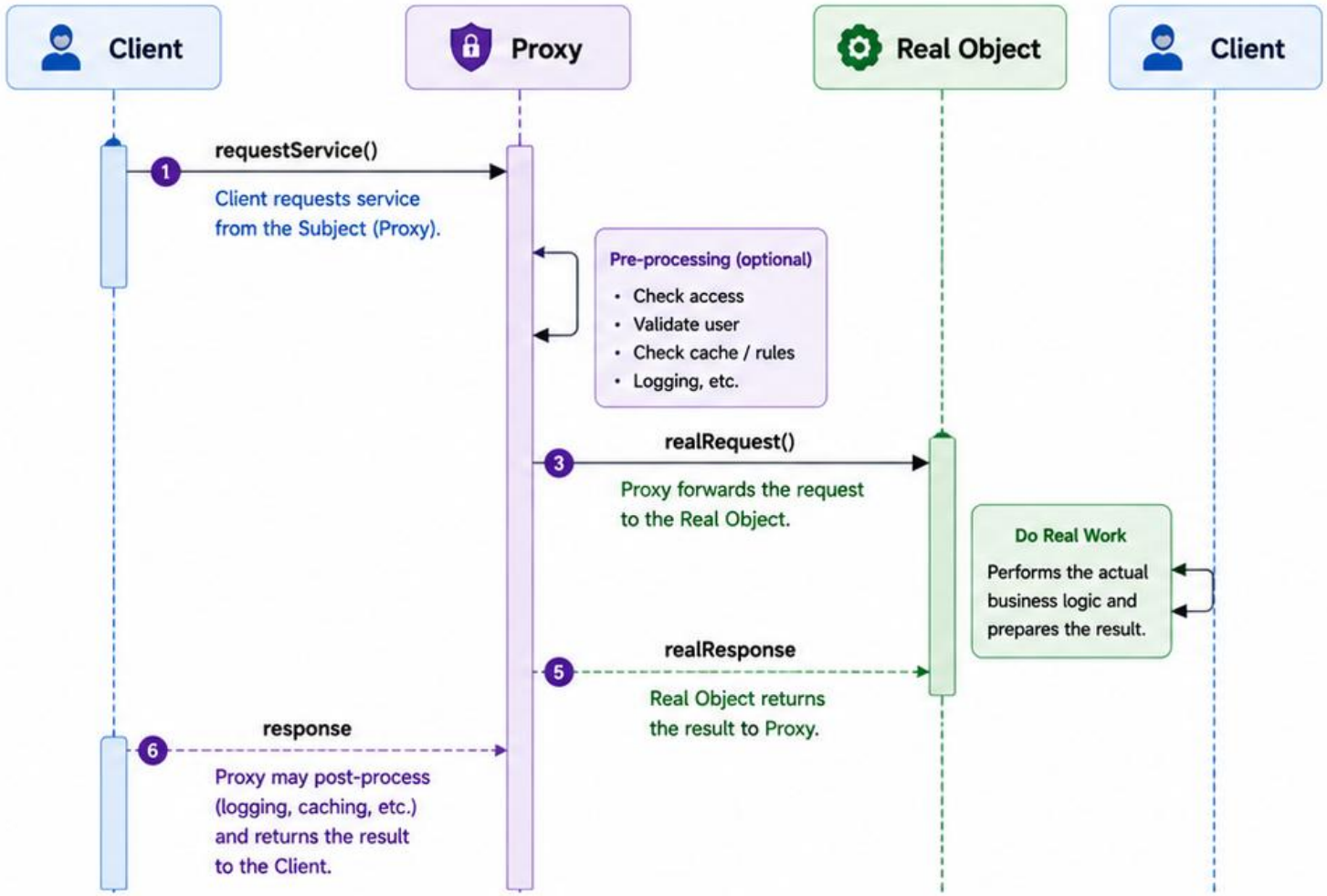
Client
Requests a service from the Subject (through Proxy).

Proxy
Controls access to the Real Object and may do extra work (checks, logging, caching, etc.).

Real Object
The actual object that performs the real work.

COMMON PROXY RESPONSIBILITIES

- Access control / authentication
- Lazy initialization
- Caching
- Logging / monitoring
- Rate limiting
- Additional business rules



FLOW SUMMARY

1. **Client → Proxy**
   Client calls a method on the Subject interface, which is actually the Proxy.
2. **Proxy (Pre-processing)**
   Proxy performs checks, access control, caching, logging, or other tasks.
3. **Proxy → Real Object**
   Proxy forwards the request to the Real Object.
4. **Real Object (Processing)**
   Real Object executes the real business logic.
5. **Real Object → Proxy**
   Real Object returns the result to the Proxy.
6. **Proxy → Client**
   Proxy may post-process the result and returns it to the Client.

KEY IDEA

The Client communicates with the Proxy, not directly with the Real Object.
The Proxy acts as an intermediary to control access, add behavior, or optimize performance.

EXAMPLE USE CASES

- **Virtual Proxy** – lazy loading (e.g., image, file, map tiles)
- **Protection Proxy** – access control (e.g., security, roles)
- **Remote Proxy** – network communication (e.g., APIs, RMI)

DISCUSSION ACTIVITY

Proxy Design Pattern – Should we use it?

Look at each situation below.
Decide: Should we use a **Proxy** or **Not**?

THINK ABOUT

- Do we need to control access?
- Do we need lazy loading?
- Do we need caching or remote access?
- Do we need to add extra logic transparently?

- USE PROXY
- DO NOT USE
- TYPE OF PROXY

#	SITUATION	DESCRIPTION	USE PROXY?	TYPE OF PROXY (If Yes)	WHY?
1	 Hospital Login System	Different users (Doctor, Nurse, Receptionist, Admin) access patient records with different permissions.	YES (Use Proxy)	Protection Proxy	Need to control access based on role and permissions.
2	 Calculator Application	Performs simple calculations locally on numbers.	NO (No Proxy)	–	No need to control access, delay creation or caching.
3	 Image Editor (High Resolution)	Large images are loaded only when the user zooms or opens the file.	YES (Use Proxy)	Virtual Proxy	Delay loading (lazy loading) of heavy images to save memory.
4	 Online Exam Authentication	Student must be verified before starting the exam.	YES (Use Proxy)	Protection Proxy	Verify identity, permissions and exam rules before access.
5	 CDN Cache for Web Content	Frequently requested content (images, CSS, JS) is served from edge servers.	YES (Use Proxy)	Cache Proxy	Proxy caches content to improve performance and reduce load.
6	 Restaurant Menu Display	Static menu is shown to customers inside the restaurant.	NO (No Proxy)	–	No need for access control, lazy loading or caching.
7	 Netflix Streaming Service	Check subscription, country restrictions, parental control before streaming.	YES (Use Proxy)	Protection Proxy (+ Cache Proxy)	Control access and stream via optimized/cached servers.
8	 Google Drive File Access	Show file list and thumbnails first. Download file only when opened.	YES (Use Proxy)	Virtual Proxy (+ Remote Proxy)	Lazy loading for large files and remote access to cloud storage.

DISCUSS & SHARE

- Why did you choose Proxy or Not for each situation?
- What type of Proxy fits best if the answer is YES?

KEY TAKEAWAY

Use Proxy when you need to control access, optimize performance, or add logic transparently without changing the real object.

BONUS QUESTION

Can you think of another real-world example where a Proxy would be useful?

PROXY DESIGN PATTERN – SUMMARY

The Proxy acts as an intermediary between the Client and the Real Subject.

Remember:
Same interface.
Different behavior.

KEY TAKEAWAYS

1. **Client always talks to the Proxy**
   The Client never communicates directly with the Real Subject.



2. **Proxy and Real Subject implement the same interface**
   This allows the Proxy to be used anywhere the Real Subject is expected.



3. **Proxy controls access**
   The Proxy decides whether to forward the request to the Real Subject.



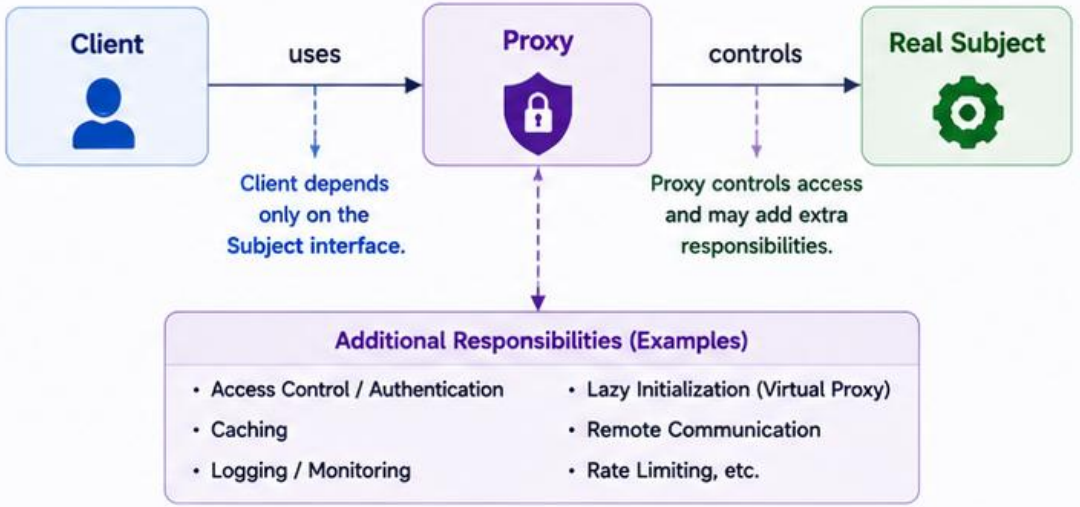
4. **Proxy may authenticate, cache, log, delay loading, or communicate remotely**
   It can add additional functionality transparently to the Client.



5. **Client usually cannot tell whether it is using the Proxy or the Real Subject**
   The substitution is transparent.



STRUCTURE (HIGH LEVEL)



WHERE PROXY IS USEFUL

- When access needs to be controlled (security, permissions)
- When object creation is expensive (lazy / on-demand loading)
- When results can be cached
- When communication is remote (network, web services)

BENEFITS

- Improves security (controlled access)
- Improves performance (caching, lazy loading)
- Hides complexity (remote access, heavy objects)
- Promotes transparency (client code stays the same)

IN ONE LINE: The Proxy looks like the Real Subject, works like a gatekeeper, and adds value without the Client knowing.